Enhancing Search Engine Relevance for Video Subtitles

Objective: Develop an advanced search engine algorithm that efficiently retrieves subtitles based on user queries, with a specific emphasis on subtitle content. The primary goal is to leverage natural language processing and machine learning techniques to enhance the relevance and accuracy of search results.

Key word-based search: Keyword-based search is a method of information retrieval where the search engine matches user queries with documents based on specific keywords or terms. In this approach, the search engine looks for exact matches or close variations of the keywords within the indexed documents and returns the documents that contain those keywords.

Here's how keyword-based search typically works:

Indexing: The search engine indexes the content of documents by analysing the text and identifying keywords or terms that are relevant to the document's content. Each document is associated with a list of keywords or terms extracted from its text.

Query Processing: When a user enters a search query, the search engine analyses the query and identifies the keywords or terms used. It may perform preprocessing steps such as removing stop words, stemming, or tokenizing the query to enhance accuracy.

Matching: The search engine compares the keywords or terms in the user query with the indexed keywords or terms associated with each document. It looks for exact matches, partial matches, or related terms to find relevant documents.

Ranking: Once the search engine identifies documents that contain the keywords or terms from the user query, it ranks the documents based on factors such as relevance, keyword density, and other relevance metrics. Documents with a higher occurrence of the keywords or terms and better match to the query are typically ranked higher.

Embedding Techniques

Bag-of-Words (BOW): In BOW representation, each document is represented as a bag of words, ignoring grammar and word order. We tokenize the text, convert it to lowercase, and remove stop words (common words like "the", "is", etc.). Then, we create a vector where each dimension represents a unique word in the corpus, and the value in each dimension represents the frequency of that word in the document. This method is simple and efficient but does not capture semantic meaning.

TF-IDF (Term Frequency-Inverse Document Frequency): TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents (corpus). It takes into account both the frequency of a term in a document (term frequency) and the rarity of the term in the entire corpus (inverse document frequency). This method helps in identifying keywords that are more discriminative and relevant to the document.

Semantic Search: Semantic-based search is a method of information retrieval that goes beyond traditional keyword matching and considers the meaning and context of words and phrases in user queries and documents. Instead of relying solely on exact keyword matches, semantic search engines aim to understand the intent behind the user's query and the content of the documents to deliver more relevant and accurate search results.

Here's how keyword-based search typically works:

Natural Language Understanding (NLU): Semantic search engines use advanced natural language processing (NLP) techniques to understand the meaning and context of user queries. This may involve parsing the query, identifying entities, relationships, and concepts, and analysing the syntactic and semantic structure of the text.

Semantic Representation: After understanding the user query, the search engine represents the query and documents in a semantic space where words and phrases are mapped to vectors or embeddings that capture their semantic meaning. This representation allows the search engine to compare the similarity between the query and documents based on their semantic content.

Semantic Matching: Using techniques such as vector similarity (e.g., cosine similarity), the search engine calculates the semantic similarity between the user query and documents in the indexed collection. Instead of relying solely on exact keyword matches, semantic matching considers the overall semantic content of the query and documents, allowing for more nuanced and accurate retrieval.

Contextual Understanding: Semantic search engines also take into account the context of the user query and documents. They may consider factors such as user preferences, location, browsing history, and social connections to personalize search results and provide more contextually relevant recommendations.

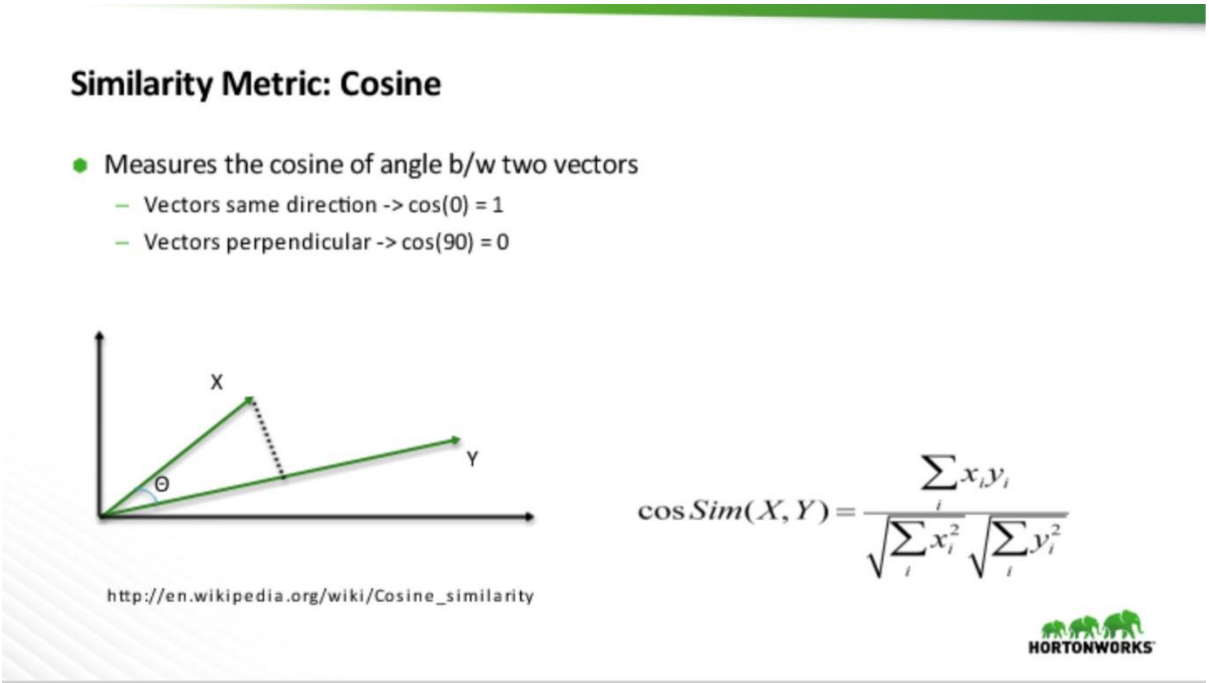
Ranking and Presentation: Finally, the search engine ranks the search results based on their semantic relevance to the user query and presents them in a user-friendly format. The most semantically similar documents are typically ranked higher, ensuring that the most relevant information is easily accessible to the user.

Semantic-based search is increasingly used in various applications, including web search engines, e-commerce platforms, recommendation systems, and enterprise search solutions. By understanding the semantics of user queries and documents, semantic search engines can deliver more accurate, personalized, and contextually relevant search results, enhancing the user experience and improving information retrieval efficiency.

Example: In Key based search if you search for laptops, it will retrieve documents only containing laptop keyword, where as in semantic search if we search for laptop, it will retrieve documents which contains laptop as well as retrieve documents which are semantically similar to laptop such as computer, phones, ultra books, tablets etc.

Cosine Similarity: Cosine similarity is a measure used to determine the similarity between two vectors in a multi-dimensional space. It calculates the cosine of the angle between the vectors, indicating how closely aligned they are in terms of direction.

Vector Representation: Each document or query is represented as a vector in a multi-dimensional space, where each dimension corresponds to a feature or term in the document. The values in the vector represent the importance or weight of each feature.



Range: The cosine similarity value ranges from -1 to 1.

A value of 1 indicates that the vectors are perfectly aligned and point in the same direction (i.e., they are identical).

A value of -1 indicates that the vectors are perfectly anti-aligned (i.e., they are exact opposites).

A value of 0 indicates that the vectors are orthogonal (i.e., they are unrelated or dissimilar).

Interpretation: In information retrieval and text processing, cosine similarity is commonly used to compare the similarity between documents or queries. Higher cosine similarity values indicate greater similarity between the documents, while lower values indicate less similarity.

Large Language Model

To convert the sentences to embedding we have used the sentence transformer 'all-MiniLM-L6-v2'. The all-* models were trained on all available training data (more than 1 billion training pairs) and are designed as general-purpose models. The all-mpnet-base-v2

model provides the best quality, while all-MiniLM-L6-v2 is 5 times faster and still offers good quality. Toggle All models to see all evaluated models or visit Hugging Face Model Hub to view all existing sentence-transformers models.

Description: All-round model tuned for many use-cases. Trained on a large and diverse dataset of over 1 billion training pairs.

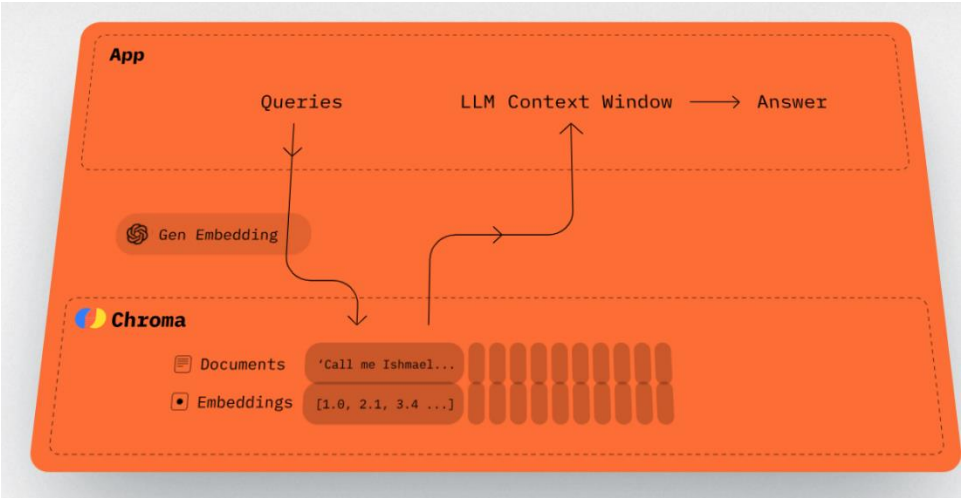
Dimensions: 768

Suitable Score Functions: dot-product (util.dot_score), cosine-similarity (util.cos_sim), euclidean distance

Size: 420 MB

Training Data: 1B+ training pairs. For details, see model card.

Chroma Database: Chroma DB is an open-source vector store used for storing and retrieving vector embeddings. Its main use is to save embeddings along with metadata to be used later by large language models. Additionally, it can also be used for semantic search engines over text data.



With Chroma DB, you can easily manage text documents, convert text to embeddings, and do similarity searches.

How It Works:

1. Ingest Documents: We ingest subtitle data, preprocess it, and apply appropriate cleaning steps.

2. Generate Embeddings: Using various vectorization methods, we generate embeddings for subtitle documents, ensuring accurate representation of content.

3. Document Retrieval: When users search for subtitles, we preprocess their queries, create query embeddings, and calculate cosine similarity scores to retrieve the most relevant documents.

Reference: https://docs.trychroma.com/

Checkout: https://github.com/t95532/Projects/tree/main/Search%20Engine